DSA Concept Sheet: TCS NQT PYQs

This sheet groups the concepts found in the document into logical categories. Each section explains the concept simply, shows how it was used in the problems, and gives you a "Golden Rule" for applying it yourself.

---

1. Mathematics & Number Theory

Often, problems that look like they require a loop (O(N) time) can be solved instantly (O(1) time) using math.

A. Arithmetic Progressions (AP) & O(1) Optimization

- **What it is:** A sequence of numbers with a constant difference (e.g., 2, 4, 6, 8...).
- **Application:** Finding the sum of even numbers up to N, or maximizing the sum of non-adjacent items in a circle.
- **How to apply:** Instead of looping through numbers to add them up, use the mathematical formula for the sum of the first natural numbers: Sum = K * (K + 1) / 2. For even numbers, it simplifies to Sum = K * (K + 1).
- 💡 **Golden Rule:** If a problem asks for the sum of a predictable sequence, **never use a loop**. Find the formula.

B. The Sieve-like GCD Approach (Harmonic Series)

- **What it is:** The Greatest Common Divisor (GCD) is the largest number that divides two numbers. Finding the maximum GCD pair among numbers by checking every pair takes O(N^2) time (too slow).
- **Application:** Finding a pair with the maximum GCD in two arrays.
- **How to apply:** Instead of checking pairs, check **potential answers**. Start from the maximum possible value and check if multiples of this value exist in the arrays. Using boolean/frequency arrays allows O(1) lookups.

- 💡 **Golden Rule:** When looking for a maximum divisor, iterate backwards from the largest possible number down to 1. The first one that satisfies your condition is your answer.

C. Manhattan Distance

- **What it is:** The distance between two points on a grid if you can only move up, down, left, and right (like a taxi driving on city blocks).
- **Application:** Minimum swaps to move an element to the center of a matrix. Swapping adjacent rows/columns is exactly moving 1 step on a grid.
- **Formula:** Distance = |row1 - row2| + |col1 - col2|
- 💡 **Golden Rule:** Anytime a problem allows "swapping adjacent grid elements," calculate the Manhattan Distance instead of actually simulating the swaps.

---

2. Array & String Traversal Techniques

A. Two-Pointer Technique (Simulated Merge)

- **What it is:** Using two variables (i and j) to point to different indices in arrays/strings to process them simultaneously without taking up extra memory.
- **Application:** Finding the median of two sorted arrays.
- **How to apply:** Instead of creating a massive third array to merge two sorted ones, place pointer i at the start of Array 1 and pointer j at the start of Array 2. Compare the elements, advance the pointer of the smaller element, and keep a count until you reach the middle index.
- 💡 **Golden Rule:** If input arrays are **already sorted**, always think "Two-Pointers" to achieve O(N) time and O(1) space.

B. Expand Around Center (Palindromes)

- **What it is:** A palindrome reads the same forwards and backwards (e.g., "racecar"). Every palindrome mirrors outward from its center.
- **Application:** Finding the longest palindromic substring.

- **How to apply:** Treat every single character (and the space between every character) as a potential center. Use two pointers (left and right) to step outward as long as the characters match.
- 💡 **Golden Rule:** A string of length N has 2N-1 possible centers (odd-length palindromes center on a letter; even-length center between letters). Check them all and track the maximum length.

---

3. Greedy Algorithms & Simulation

A. Greedy Strategy

- **What it is:** Making the locally optimal ("best right now") choice at each step with the hope that it leads to the global optimum. You don't look back or re-evaluate.
- **Application:**
  - *Project Selection:* Pick the project with the highest profit that fits this year's budget.
  - *Task Scheduler:* Always cook the dish that has the highest remaining count and is not on cooldown.
- **How to apply:** Sort the data based on your goal (e.g., highest profit, highest frequency), then iterate through and pick the first valid option.
- 💡 **Golden Rule:** Use Greedy when choices don't negatively impact future choices. If choosing A now ruins your chance to choose B later, you might need Dynamic Programming instead.

B. Simulation with Hash Maps

- **What it is:** Literally mimicking the rules of the problem step-by-step using a time counter.
- **Application:** The Kitchen Task Scheduler (managing cooldowns).
- **How to apply:** Use a Hash Map (Dictionary) to track the frequency of items, and a second Hash Map to track the last_used_time. At time T, check if T - last_used_time > cooldown.

- 💡 **Golden Rule:** Hash Maps provide O(1) constant time lookups. Always use them when tracking "How many?" (frequencies) or "When?" (timestamps).

---

4. Bit Manipulation

A. Bitwise Independence & XOR Properties

- **What it is:** In binary, numbers are represented as 0s and 1s. The XOR operation (^) returns 1 if the bits are different, and 0 if they are the same. **Crucial insight:** What happens at the 1st bit position has absolutely no effect on the 2nd bit position. They are independent.
- **Application:** Maximizing an XOR sum with an array.
- **How to apply:** Instead of looking at the whole numbers, look at them in "vertical columns" of bits. For the j-th bit, count how many 0s and 1s exist in the array.
  - If you set your answer's bit to 1, all 0s become 1s, and 1s become 0s.
  - *Gain in sum* = (Count of 0s - Count of 1s) * 2^j.
- 💡 **Golden Rule:** When a problem asks to maximize/minimize an XOR/AND/OR sum, break the numbers down and analyze the problem **bit by bit** (usually from bit 0 to bit 30/31).

---

5. Dynamic Programming (DP)

A. The "Coin Change" Variation

- **What it is:** DP is used when a problem asks for the "minimum/maximum ways" to reach a target, and making a Greedy choice doesn't guarantee the correct answer. It works by solving smaller sub-problems and storing the answers.
- **Application:** Finding the minimum digits needed to form a remainder (Minimum N-Digit Perfect Square Sum).
- **How to apply:** Create an array dp[] initialized to infinity. dpstores the minimum items needed to make sum x. If you have a list of possible increments, the

transition is: dp[current_sum] = min(dp[current_sum], dp[current_sum - increment] + 1).

- 💡 **Golden Rule:** If you need to hit a precise target sum using minimum elements, and the elements are varied sizes (like 3, 8, 15), use DP, not Greedy.

---

6. Crucial "Hidden" Concepts to Remember

1. **Large Integer Arithmetic (Overflow):** Standard 32-bit integers max out around 2 * 10^9. If a problem has inputs up to 10^9 and asks you to multiply or sum them, you **will** exceed this limit.
   - *Solution:* Always use long in Java, long long in C++, or arbitrary precision types (default in Python).
2. **Constraint Satisfaction (Bounding):** Sometimes checking every number is too slow.
   - *Solution:* Find the upper and lower mathematical bounds. In the "Decrypting the Message" problem, instead of testing every key, the code deduces that the Key *must* be ≤ (Smallest Encrypted Value - 1) and ≥ (Largest Encrypted Value - 26).
3. **ASCII Character Arithmetic:** Characters are stored as numbers in computers.
   - *Solution:* You can do math on letters. In Python: ord('C') - ord('A') + 1 = 3. In C++/Java: 'C' - 'A' + 1 = 3. Use this to map letters to numbers and vice versa instantly.